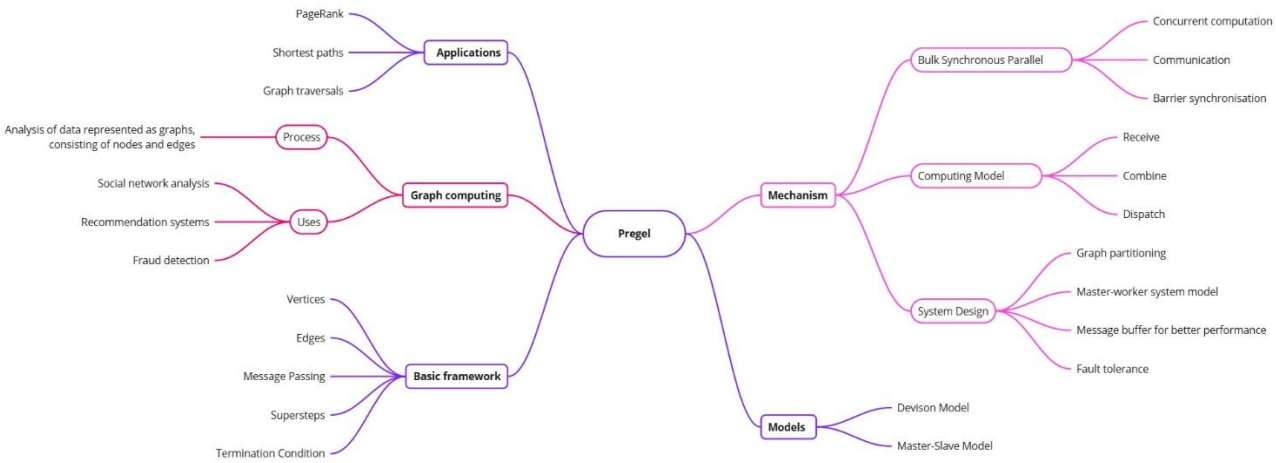
Experiment Report

Student: Denisenko Sofiia 狄苏菲
Student id: 1820249051
Course: Big Data Analysis Technology

Experiment Topic: Pregel

Mind Map Pregel



Questions

1. What happened in Graph level, partition level and node level when using Pregel do Graph Computation?

   Answer:

   Graph level

   Pregel operates on a distributed graph, so it is stored across multiple machines. The graph is partitioned into smaller subgraphs, which are assigned to different compute nodes. At the start of each superstep, every node in the graph receives its messages from the previous superstep. Each node then performs its computation, sending messages to its neighbors. The superstep ends when all nodes have completed their computation and sent their messages.

   Partition level

   Each partition works independently during a superstep. The partitions can communicate with each other to exchange messages between their nodes. Each partition contains a vertex manager responsible for maintaining the data and operations for the vertices within the partition.

   Node level

   Individual nodes execute their computation based on their received messages and current state. They can update their state and send messages to neighboring nodes. The computation at each node is performed by a user-defined vertex program, which specifies the node's logic.

2. How can we divide the graph into partitions?

   Answer:

   In the Pregel computation framework, a large graph is divided into multiple partitions, each of which contains a subset of vertices and the edges that originate from them. Several techniques are used to divide the graph into partitions, aiming for balanced workload distribution:

   - **Hash partitioning:** The most straightforward approach is to apply a hash function to each vertex's ID. The hash function determines which partition a vertex belongs to. This is simple to implement but might not be optimal for graphs with uneven vertex distributions.
   - **Range partitioning:** Assigning vertices to partitions based on their IDs falling within specific ranges. For example, vertices with IDs 1-1000 might go to partition 1, IDs 1001-2000 to partition 2, and so on.

- **Graph partitioning algorithms:** More sophisticated algorithms like METIS, KaHIP, and Graclus aim to minimize the number of edges cut across partitions. This helps to reduce communication overhead during Pregel computations. These algorithms consider the graph structure and try to balance workload and minimize communication.

The ideal partitioning strategy depends on the graph's structure and the computation workload.

3. Explain the Fault Tolerance mechanism in Pregel.

Answer:
Pregel employs several mechanisms to handle node failures:

Checkpoints are saving the state of all nodes and their messages.

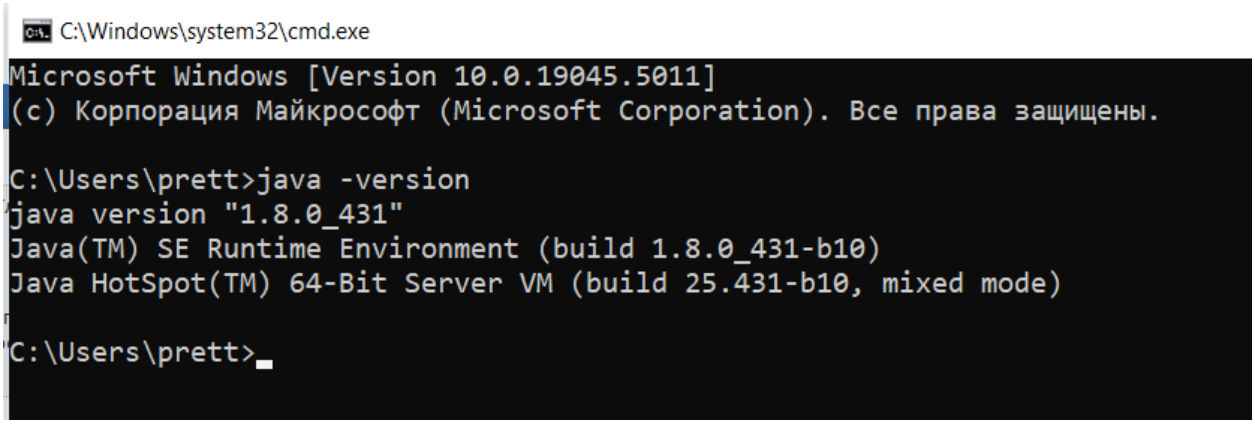
Partitions are replicated on different nodes to provide redundancy.

Messages are buffered in queues, and if a node fails, the messages are rerouted to the other replica of the partition.

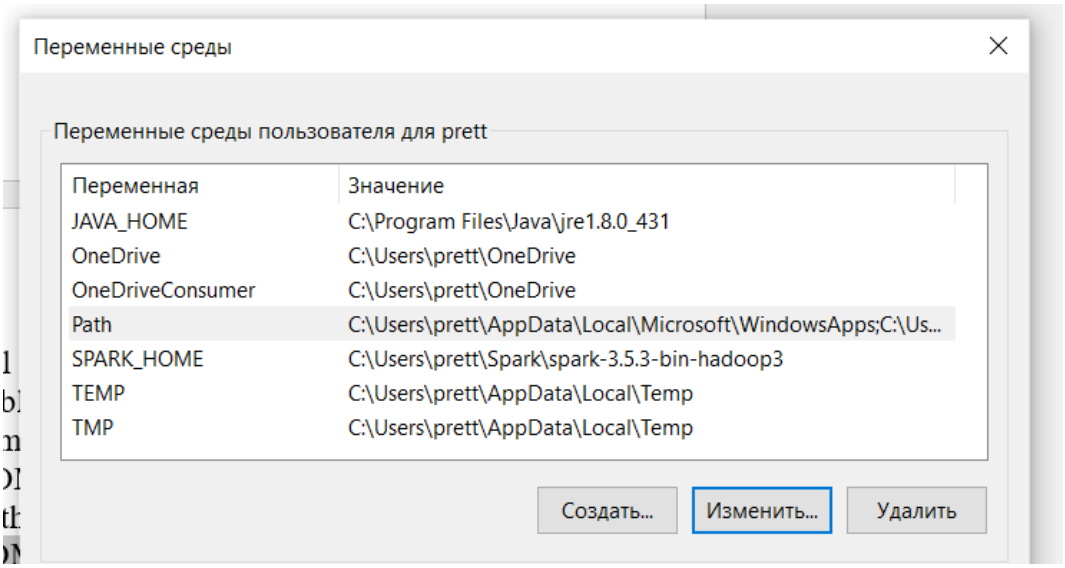
When a node recovers, it loads its state from the checkpoint and begins processing messages from its queue.

Preparations for the Experiment

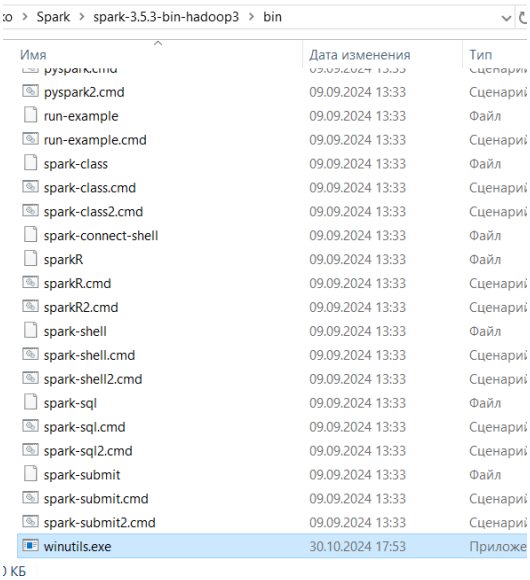
At first, I downloaded Java 8 from the original website.



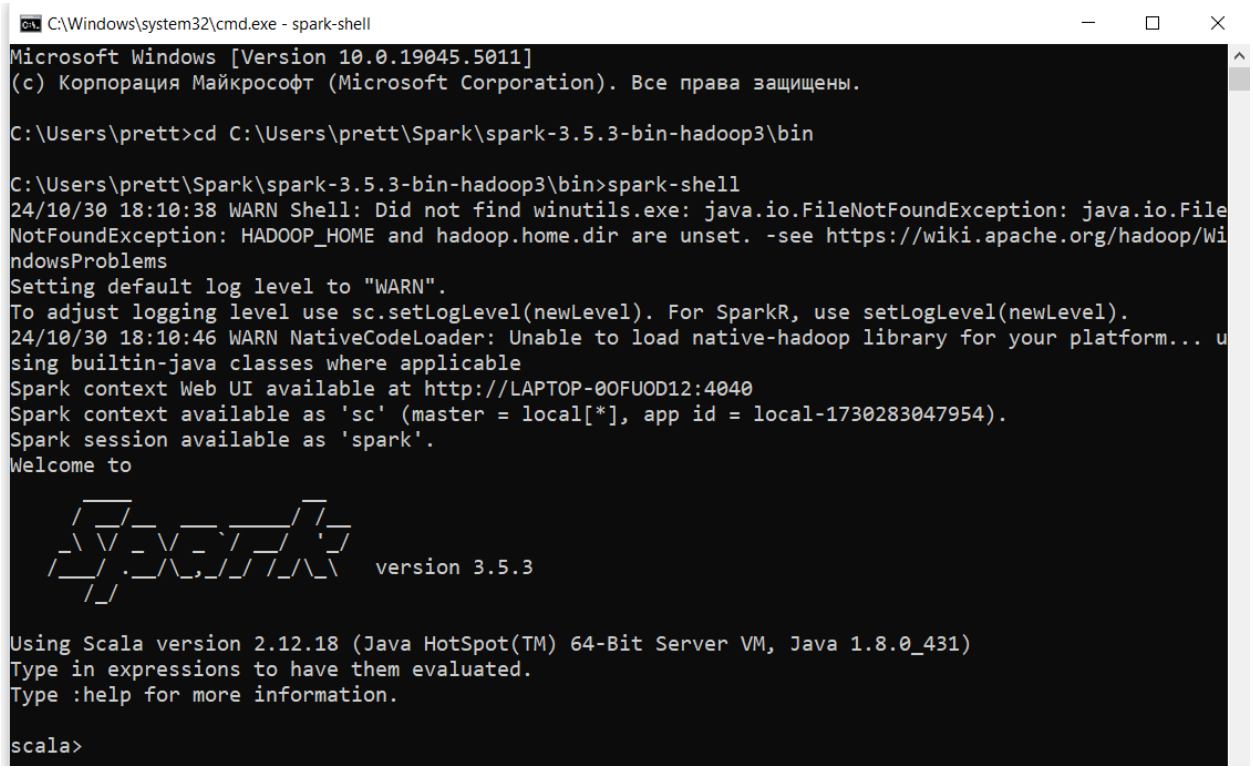
Downloading Spark and adding environmental variables (JAVA_HOME) and (SPARK_HOME)



Downloading winutils and adding to the Spark directory.



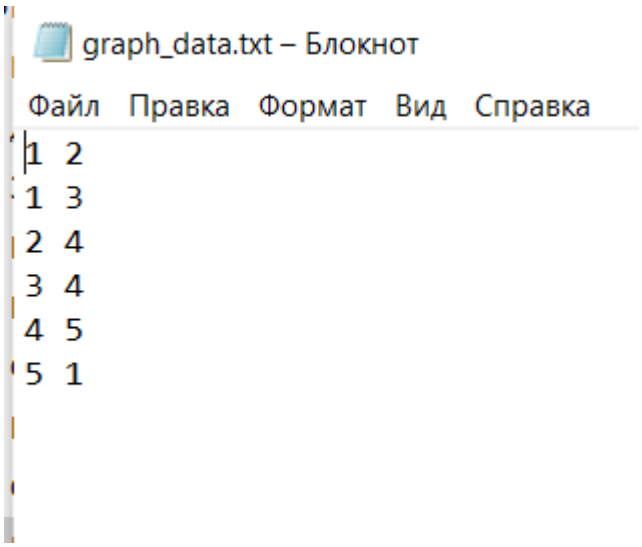
Starting Spark-shell.



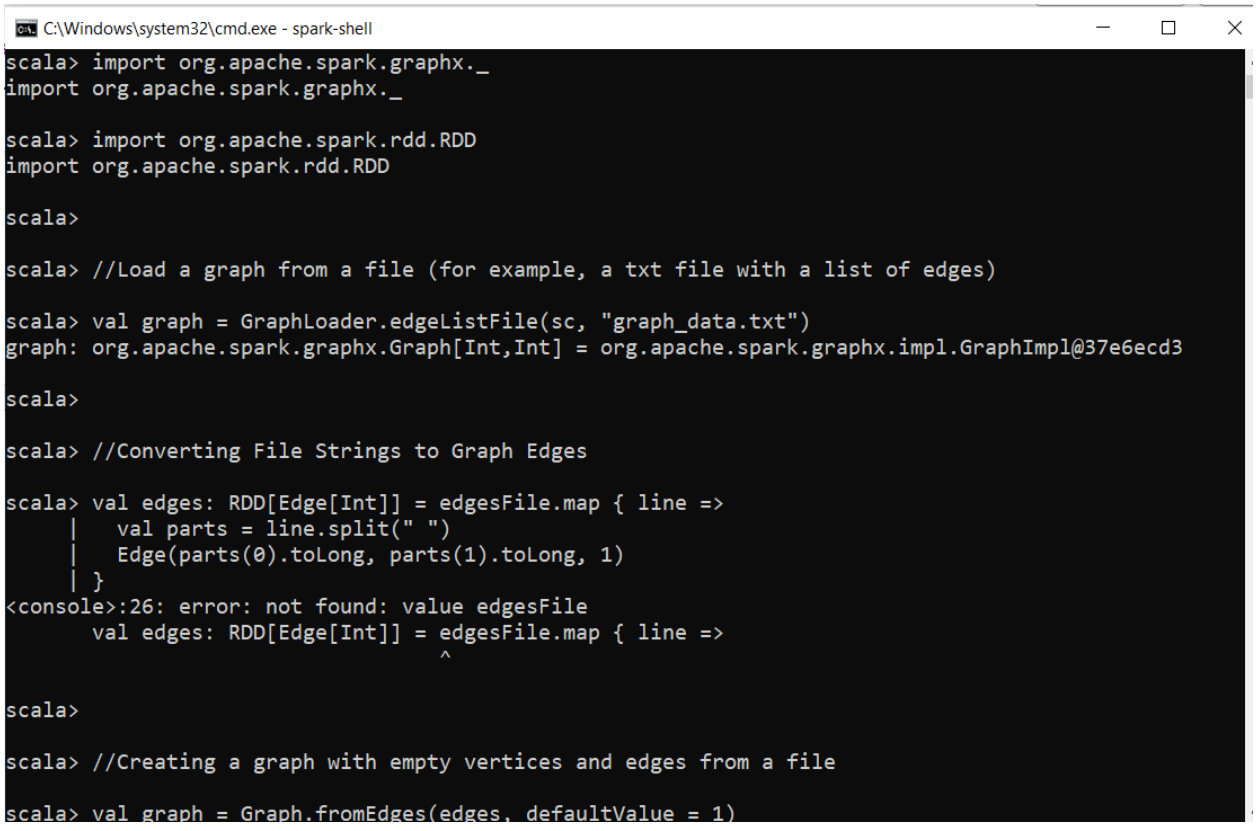
Experiment 1

Experiment Topic: Working with the File

Content of “graph_data.txt”



Pasting the code and running it.



Although there were some errors in compilation, the code showed correct results.

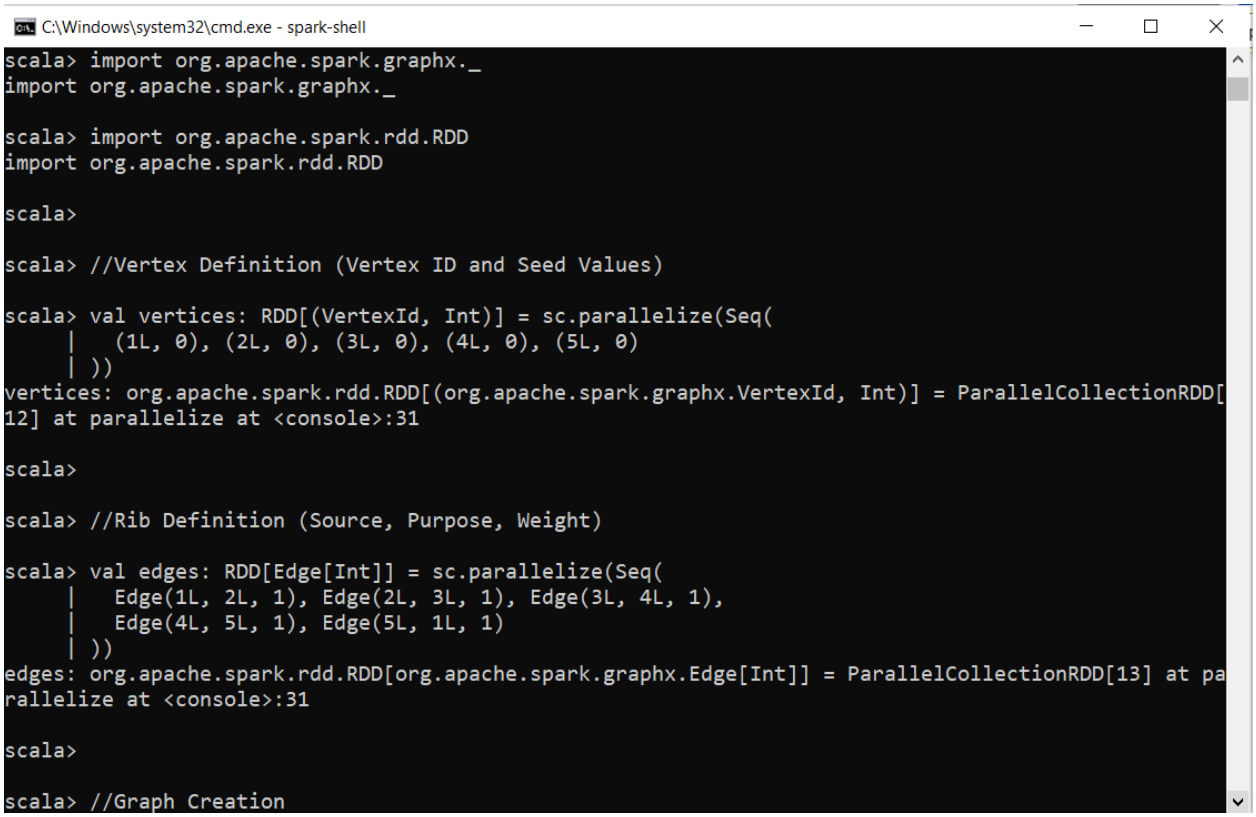
```
scala> graph.vertices.collect.foreach(println)
(4,1)
(2,1)
(1,1)
(3,1)
(5,1)

scala> graph.edges.collect.foreach(println)
Edge(1,2,1)
Edge(1,3,1)
Edge(2,4,1)
Edge(3,4,1)
Edge(4,5,1)
Edge(5,1,1)
```

Analysis: By running this code, we collected the list of vertex and edges from the text file.

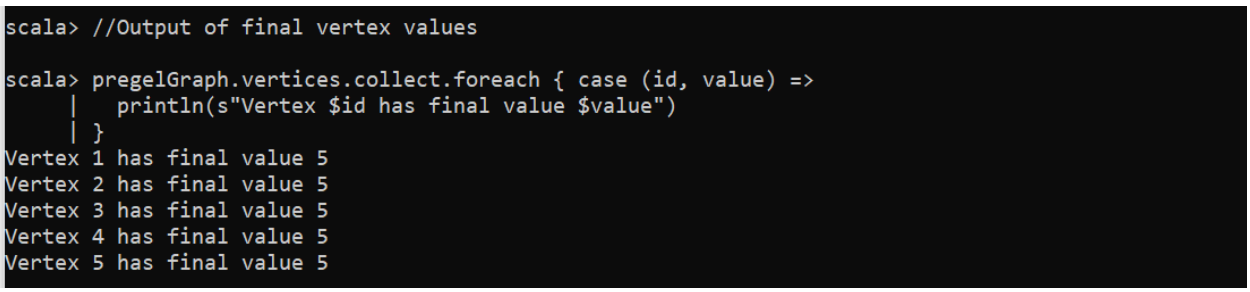
Experiment 2

Experiment Topic: Propagation of Pregel Values

Pasting the code and running it.



```
C:\Windows\system32\cmd.exe - spark-shell
scala> import org.apache.spark.graphx._
import org.apache.spark.graphx._

scala> import org.apache.spark.rdd.RDD
import org.apache.spark.rdd.RDD

scala>

scala> //Vertex Definition (Vertex ID and Seed Values)

scala> val vertices: RDD[(VertexId, Int)] = sc.parallelize(Seq(
     |   (1L, 0), (2L, 0), (3L, 0), (4L, 0), (5L, 0)
     | ))
vertices: org.apache.spark.rdd.RDD[(org.apache.spark.graphx.VertexId, Int)] = ParallelCollectionRDD[
12] at parallelize at <console>:31

scala>

scala> //Rib Definition (Source, Purpose, Weight)

scala> val edges: RDD[Edge[Int]] = sc.parallelize(Seq(
     |   Edge(1L, 2L, 1), Edge(2L, 3L, 1), Edge(3L, 4L, 1),
     |   Edge(4L, 5L, 1), Edge(5L, 1L, 1)
     | ))
edges: org.apache.spark.rdd.RDD[org.apache.spark.graphx.Edge[Int]] = ParallelCollectionRDD[13] at pa
rallelize at <console>:31

scala>

scala> //Graph Creation
```



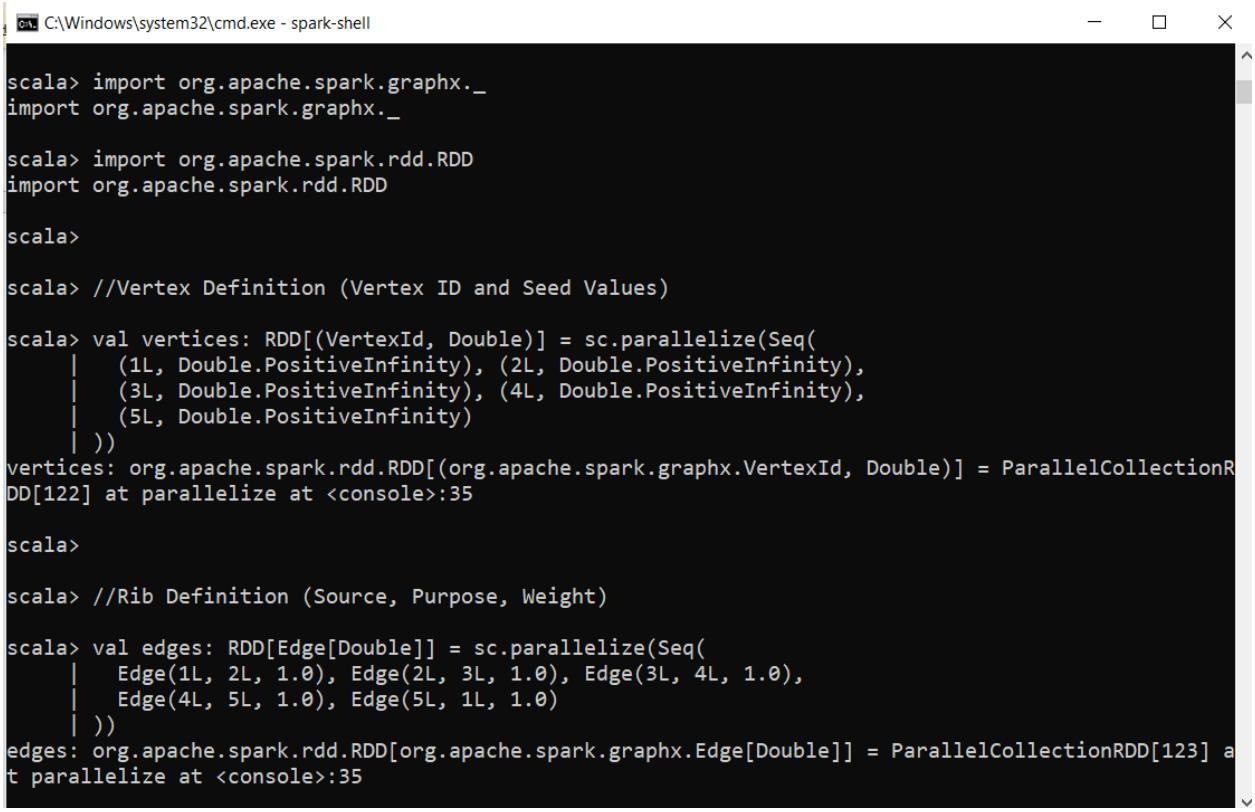
```
scala> //Output of final vertex values

scala> pregelGraph.vertices.collect.foreach { case (id, value) =>
     |   println(s"Vertex $id has final value $value")
     | }
Vertex 1 has final value 5
Vertex 2 has final value 5
Vertex 3 has final value 5
Vertex 4 has final value 5
Vertex 5 has final value 5
```

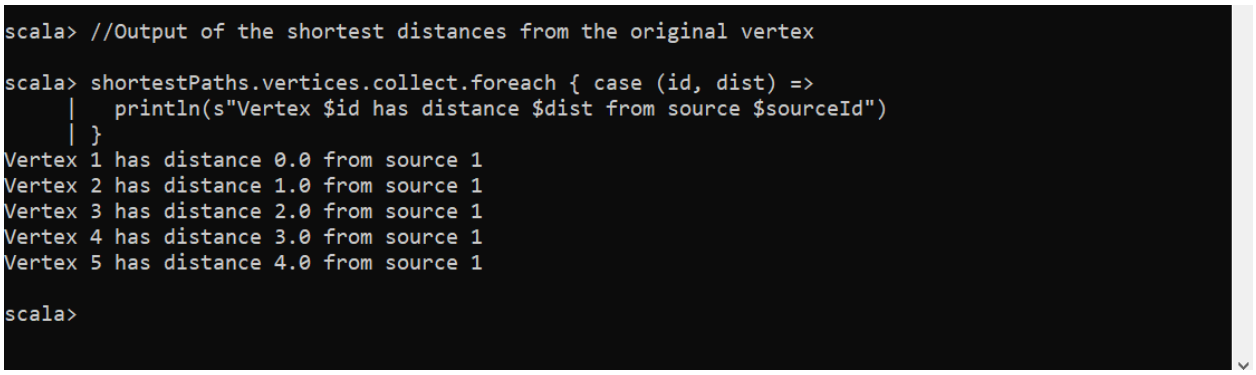
Analysis: In each iteration, the vertices send new messages to their neighbors based on their values. The vertex value continues to increase until it reaches the maximum value of messages received (5). In the end we ptinted the vertex values.

Experiment 3

Experiment Topic: Finding the minimum distance to a given vertex

Pasting and running the code.

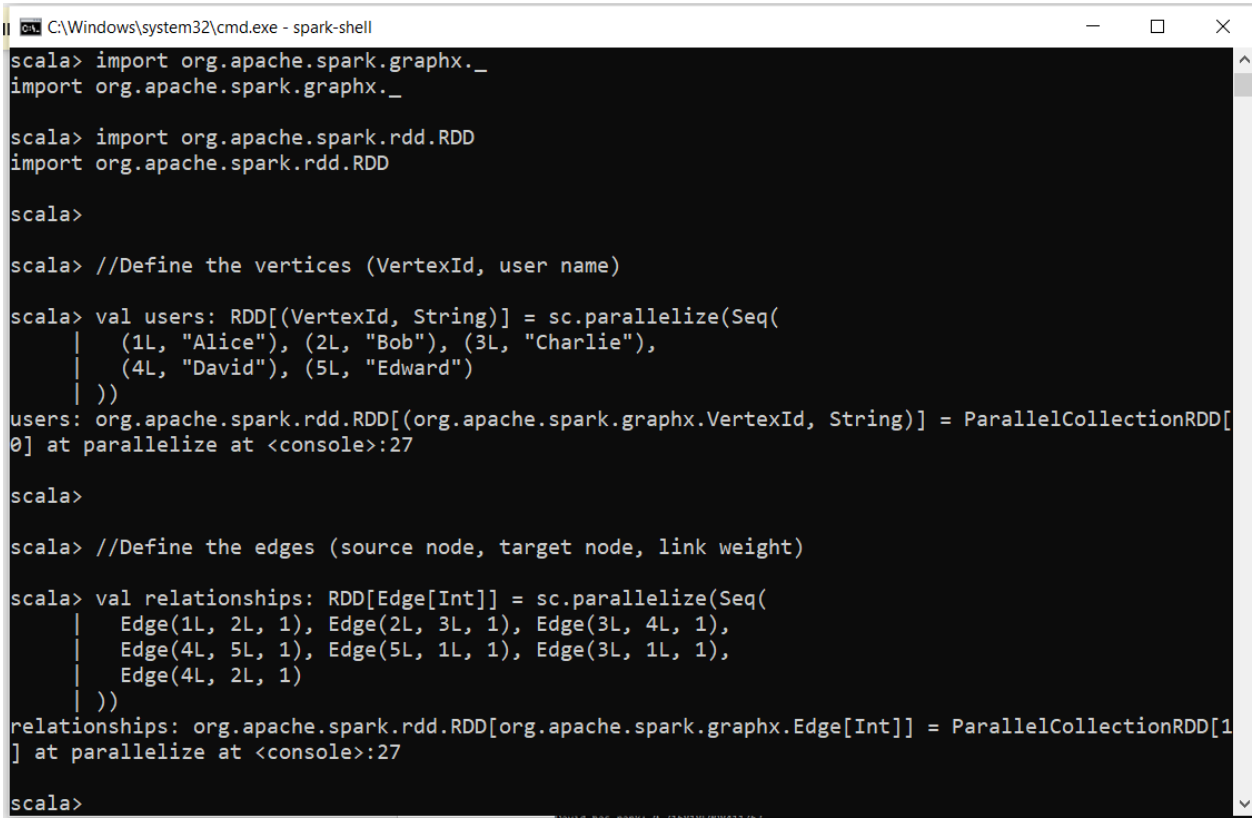


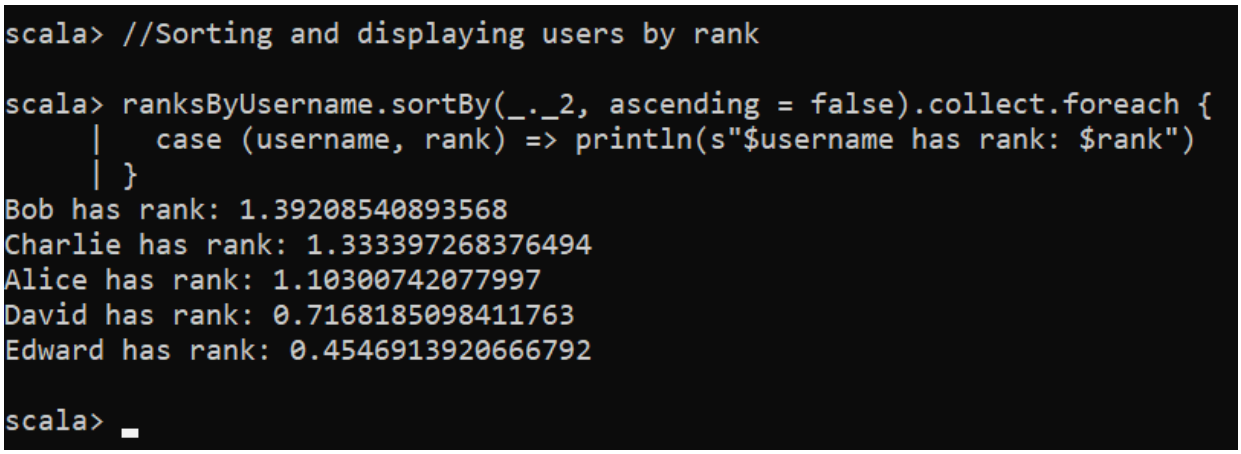


Analysis: This code implements the classic single-source shortest path algorithm using the Pregel framework in Apache Spark. It finds the shortest distances from a given source vertex to all other vertices in a graph.

Experiment 4

Experiment Topic: User Relationship Analysis





Analysis: This Spark code implements the PageRank algorithm to calculate the importance of each user in a social network represented by a graph. Bob has the highest rank so he is the most important person in this network.